Adapter Pattern for Music Band

Unifying Musical Instrument Interfaces

Prepared by:
[Names]

Course: Design Patterns

Agenda

- Real-world Problem: Different instrument interfaces
- Design Challenges & Trade-offs (Adapter vs. Facade)
- Why Adapter? Justification & Design Principles
- UML Class Diagram
- Code Structure & Walkthrough
- Live Demo (run Main.java)
- Team Roles & GitHub Contributions
- Summary of Deliverables (C1 & C2)
- Conclusion & Q&A

Real-World Problem

- Real-world software problem:
- In a music band management system, different musical instruments expose different methods to produce sound. For example:
- Electric guitar requires startElectric() then playSound().
- Electric piano requires startElectric() then playSound() (similar but not identical).
- Flute requires blow().
- Drum requires hit().
- A musician or conductor wants to play any instrument using a single, unified command (e.g., play()) without knowing each instrument’s specific interface.

Main design challenges

- Interface incompatibility between existing instrument classes.
- The system must be open for adding new instrument types (e.g., saxophone, violin) without modifying existing client code.
- The client (musician) should be decoupled from concrete instrument implementations.

Comparison of two possible design patterns:

Intent	Convert interface of an existing class into another interface expected by client.	Provide a simplified interface to a complex subsystem.
Solves	Interface mismatch.	Complexity and many dependencies.
Application	Wrap each instrument individually.	Wrap the whole audio subsystem.
In our problem	✅ Directly adapts each instrument's unique methods to play().	❌ Does not solve interface mismatch; only hides complexity.
Trade-offs	Slight increase in number of classes (adapters). Very flexible.	Simpler but cannot make blow() become play().

Selected pattern

- Selected pattern: Adapter Pattern (Object Adapter variant).

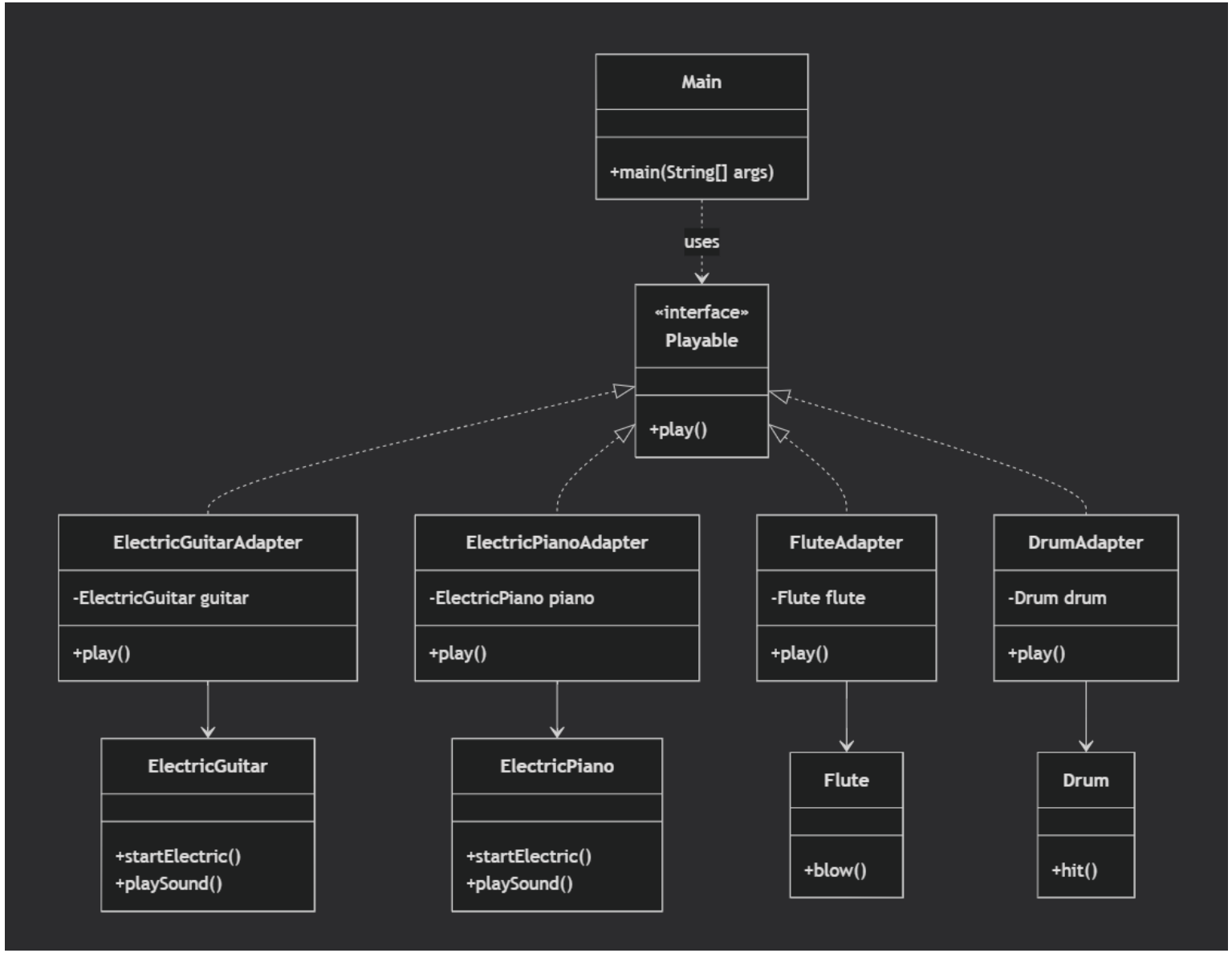
- Justification: It directly solves the interface mismatch, preserves the Open/Closed Principle, and allows reuse of existing instrument classes without modification.

Pattern Selection & Justification Document

- Why it fits the problem:
- The problem is a classic “interface incompatibility” scenario.
- Adapter allows the client to call play() on any instrument, while internally calling startElectric() + playSound(), blow(), or hit() as needed.
- It enables adding new instruments later by just writing a new adapter – no change to client code.

Reasoning based on design principles

- Reasoning based on design principles:
- Single Responsibility Principle: Each adapter is responsible for adapting exactly one type of instrument.
- Open/Closed Principle: The system is open for extension (new adapters) but closed for modification (client code remains unchanged).
- Interface Segregation Principle: The target interface Playable is minimal (only play()).
- Dependency Inversion: Client depends on abstraction (Playable), not on concrete instrument classes.


Main
+main(String[] args)
uses
«interface»
Playable
+play()
ElectricGuitarAdapter
-ElectricGuitar guitar
+play()
ElectricPianoAdapter
-ElectricPiano piano
+play()
FluteAdapter
-Flute flute
+play()
DrumAdapter
-Drum drum
+play()
ElectricGuitar
+startElectric()
+playSound()
ElectricPiano
+startElectric()
+playSound()
Flute
+blow()
Drum
+hit()

UML Diagram Technical Documentation

- ## Diagram Overview
- The attached UML class diagram illustrates the Adapter Pattern applied to unify musical instrument interfaces.

- ## Elements
- - **Target Interface (Playable)**: Declares the unified `play()` method.
- - **Adaptees (ElectricGuitar, ElectricPiano, Flute, Drum)**: Existing classes with incompatible interfaces (`startElectric()+playSound()`, `blow()`, `hit()`).
- - **Adapters (ElectricGuitarAdapter, etc.)**: Each implements `Playable` and delegates `play()` to the adaptee's specific methods. They also hold a reference to their respective adaptee (association).
- - **Client (Main)**: Depends on `Playable` only (dependency arrow labeled "uses").

- ## How the Pattern is Applied
- - The client calls `play()` on any `Playable` object.
- - Each adapter translates `play()` into the correct adaptee method(s).
- - New instruments can be added by creating new adapter classes without modifying existing code (Open/Closed Principle).

- ## Relationships
- - **Implements (dashed arrow with hollow triangle)**: From each adapter to `Playable`.
- - **Association (solid arrow)**: From each adapter to its corresponding adaptee.
- - **Dependency (dashed arrow)**: From `Main` to `Playable` (labeled "uses").

How the pattern is applied:

- How the pattern is applied:
- Target: Playable interface with play().
- Adaptees: ElectricGuitar, ElectricPiano, Flute, Drum – each has its own method(s).
- Adapters: Each adapter implements Playable and delegates to the adaptee’s specific methods.
- Client: Main class (or Musician) works with Playable only.

Pattern Implementation – Source Code + README

- src/
- ├── Playable.java
- ├── adaptees/
- │ ├── ElectricGuitar.java
- │ ├── ElectricPiano.java
- │ ├── Flute.java
- │ └── Drum.java
- ├── adapters/
- │ ├── ElectricGuitarAdapter.java
- │ ├── ElectricPianoAdapter.java
- │ ├── FluteAdapter.java
- │ └── DrumAdapter.java
- └── Main.java